

Outline of the lecture

- **CODES**
 - **SEQUENTIAL CODES**
 - **SELF-COMPLEMENTING CODES**
 - **CYCLIC CODES**
 - **WEIGHTED CODES**
 - **8421 OR BCD CODES**
 - **2421 CODES**
 - **NON-WEIGHTED CODES**
 - **EXCESS-3 (XS-3) CODES**
 - **GRAY CODES**
 - **CODE CONVERSION**
 - **GRAY CODES**
 - ✓ 4- Bit Binary to Gray code conversion
 - ✓ 4-Bit Gray to Binary code conversion
 - **BCD CODES**
 - ✓ BCD to Excess-3 code conversion
 - ✓ 4- Bit Binary to BCD code conversion
 - **APHANUMERIC CODES**
 - ✓ ASCII codes
 - ✓ EBCDIC codes
- **PROBLEMS FOR PRACTICE**

CODES

- ▶ Code is a symbolic representation of discrete information which may be present in form of numbers, alphabets or physical quantities.
- ▶ ENCODING means to represent numbers, letters or words by a special group of symbols called CODE.
- ▶ Codes can be classified as:
 - ▶ Weighted Binary codes
 - ▶ Non-weighted codes
 - ▶ Alphanumeric codes
 - ▶ Error-detecting codes
 - ▶ Error- correcting codes

CODES

- ▶ **Sequential Code**: Codes in which each succeeding code word is one binary greater than its preceding code word.
 - ▶ 8421 and Excess-3
- ▶ **Self- Complementing Code**: If the code word of 9's complement of N ($9-N$) is obtained by interchanging all 0s and 1s of N .
 - ▶ 2421, 5211, EXCESS-3 are self-complementing code
 - ▶ For a code to be self-complementing, the sum of all the weights should be 9
- ▶ **Cyclic code**: Codes in which each successive code differs from its preceding code by only one bit position.
 - ▶ Gray code is a cyclic code

WEIGHTED BINARY CODES

- ▶ These codes obey positional weighting principle
 - ▶ Each position of a number represents a specific weight.
 - ▶ Each digit is multiplied by their respective weights and the products are summed to get the equivalent decimal number.
- ▶ Types of weighted codes:
 - ▶ 8421 Code
 - ▶ 2421 Code
 - ▶ 5421 Code
 - ▶ 5211 Code
 - ▶ 3321 Code
 - ▶ 4311 Code

WEIGHTED BINARY CODES

► 8421 weighted codes:

- Each decimal digit is represented by a 4-bit binary number
- From left to right, the weights are 8, 4, 2, 1
- Also called Binary Coded Decimal (BCD) 8421 Code

Weights	8	4	2	1
Number = 9	1	0	0	1
Number = 4	0	1	0	0

- In total only 10 valid codes are there, each for digit 0 to 9
- E.g. 12 in binary is (1100) but in BCD 12 is (0001 0010)
9 in binary is (1001) and in BCD 9 is also (1001)
- 1010, 1011, 1100, 1101, 1110, 1111 are six invalid combinations in this code
- Advantage: Easy to convert to and from decimal
- Disadvantage: requires more number of bits

WEIGHTED BINARY CODES

► 2421 weighted codes:

- Each decimal digit is represented by a 4-bit binary number
- From left to right, the weights are 2, 4, 2, 1
- In total only 10 valid codes are there, each for digit 0 to 9
- It is a Self-complementing Code

Weights	2	4	2	1
Number = 3	0	0	1	1
Number = 5	1	0	1	1

- E.g. 2421 code of 3 = 0011; 9's complement of 3 is 6
2421 code of 6 = 1100 {interchange 0s and 1s of 0011(3)}

NON-WEIGHTED BINARY CODES

- ▶ These codes are not positionally weighted
 - ▶ no fixed value of weight is assigned to each position
- ▶ Types of non-weighted codes:
 - ▶ EXCESS-3 (XS-3) Code
 - ▶ Gray Code

NON-WEIGHTED BINARY CODES

► EXCESS-3 (XS-3) codes:

- The code for each digit is obtained by adding 3(0011) to its corresponding BCD code
- It is a sequential code
- It is a self-complementing code
- Useful in arithmetic operations
- 0000, 0001, 0010, 1101, 1110, 1111 are six invalid combinations in this code
- E.g. XS-3 code of 3 = $3 + 3 = 6 = (0110)_2$

OR

$$\text{XS-3 code of 3} = 0011 + 0011 = 0110 = (6)_{10}$$

- E.g. XS-3 code of 5 = $5 + 3 = 8 = 1000$

NON-WEIGHTED BINARY CODES

► Gray codes:

- It is a minimum change code, as two consecutive codes differ at only one bit position. Hence, also called as UNIT DISTANCE CODE
- It is a cyclic code
- It is a reflective code

Decimal	Gray code	
0	0	0
1	0	1
2	1	1
3	1	0

Decimal	Gray code		
0	0	0	0
1	0	0	1
2	0	1	1
3	0	1	0
4	1	1	0
5	1	1	1
6	1	0	1
7	1	0	0

CODE CONVERSIONS

- ▶ Presence of large variety of codes results in usage of different codes by different digital systems
- ▶ Conversion among the codes allow the two systems to be compatible, even though each system was using a different binary code
- ▶ Steps for code conversion from binary code 'A' to binary code 'B':
 - ▶ Step 1: Formulate the truth table for the desired conversion
 - ▶ The input lines in the truth table comprise of all possible bit combination of elements in code 'A'
 - ▶ The output lines in the truth table comprise of corresponding bit combinations generated in code 'B'
 - ▶ Step 2: Simplify the Boolean function for each output variable in terms of minterms/maxterms
 - ▶ Step 3: Minimize the Boolean function in SOP/POS using K-map
 - ▶ Step 4: Realize the converter circuit using desired logic gates.
 - ▶ The inputs of the circuit will represent the variables of code 'A'
 - ▶ The output of the circuit will represent variables of code 'B'

CODE CONVERSIONS-BINARY TO GRAY

4-bit Binary code to 4-bit Gray code:

- ▶ The input is 4-bit Binary number, hence 16 input combination are there
 - ▶ The input variables are: B_3 , B_2 , B_1 , and B_0
 - ▶ B_3 is the MSB and B_0 is the LSB
- ▶ The output is 4-bit number in Gray code, one for each of the 16 Binary combination
 - ▶ The output variables are: G_3 , G_2 , G_1 , and G_0
 - ▶ G_3 is the MSB and G_0 is the LSB

CODE CONVERSIONS-BINARY TO GRAY

4-bit Binary code to 4-bit Gray code: The input is 4-bit Binary number and output is 4-bit number in Gray code. Truth table is (STEP 1)

minterm	4-bit Binary				4-bit Gray			
	B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0

CODE CONVERSIONS-BINARY TO GRAY

4-bit Binary code to 4-bit Gray code:

(STEP 2)

- ▶ From the truth table, the equation for each output variable in terms of minterms are
 - ▶ $G_3 = \sum m (8, 9, 10, 11, 12, 13, 14, 15)$
 - ▶ $G_2 = \sum m (4, 5, 6, 7, 8, 9, 10, 11)$
 - ▶ $G_1 = \sum m (2, 3, 4, 5, 10, 11, 12, 13)$
 - ▶ $G_0 = \sum m (1, 2, 5, 6, 9, 10, 13, 14)$

- ▶ The reduced SOP expression for G_3 , G_2 , G_1 AND G_0 is calculated using K-map

CODE CONVERSIONS-BINARY TO GRAY

4-bit Binary code to 4-bit Gray code: Using K-maps, the minimal expression for output variables G_3 , G_2 , G_1 , and G_0 , is obtained in terms of input variables, B_3 , B_2 , B_1 , and B_0 . (STEP 3)

		B_1B_0			
		00	01	11	10
B_3B_2	00	0	0	0	0
	01	0	0	0	0
	11	1	1	1	1
	10	1	1	1	1

$$G_3 = B_3$$

		B_1B_0			
		00	01	11	10
B_3B_2	00	0	0	0	0
	01	1	1	1	1
	11	0	0	0	0
	10	1	1	1	1

$$G_2 = B_3'B_2 + B_3B_2' = B_3 \text{ XOR } B_2$$

		B_1B_0			
		00	01	11	10
B_3B_2	00	0	0	1	1
	01	1	1	0	0
	11	1	1	0	0
	10	0	0	1	1

$$G_1 = B_2'B_1 + B_2B_1' = B_2 \text{ XOR } B_1$$

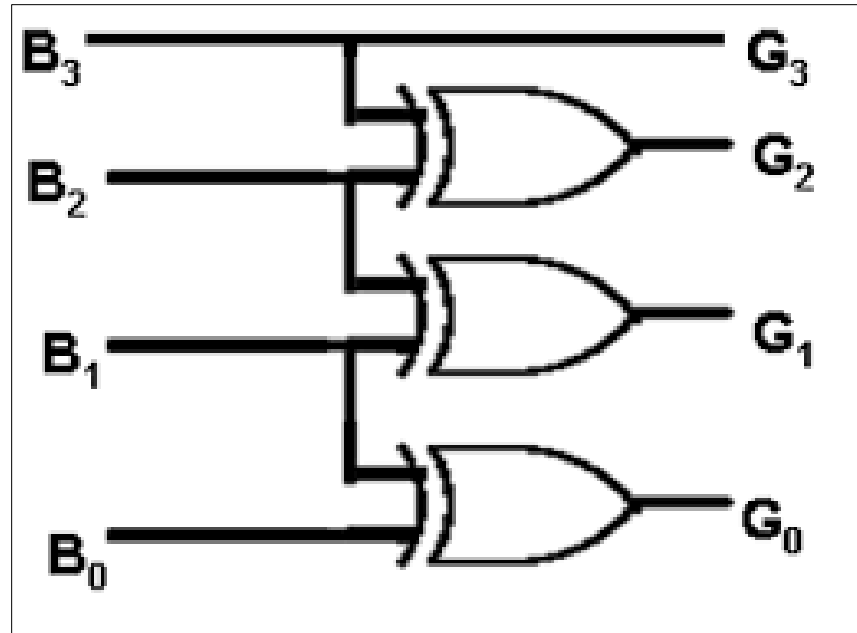
		B_1B_0			
		00	01	11	10
B_3B_2	00	0	1	0	1
	01	0	1	0	1
	11	0	1	0	1
	10	0	1	0	1

$$G_0 = B_1'B_0 + B_1B_0' = B_1 \text{ XOR } B_0$$

CODE CONVERSIONS-BINARY TO GRAY

- ▶ The logic gate realization of 4-bit Binary to Gray code converter requires 3 XOR gates.
- ▶ The logic diagram for the converter is

(STEP-4)



CODE CONVERSIONS-GRAY TO BINARY

4-bit Gray code to 4-bit Binary code: The input is a 4-bit Gray code number and output is 4-bit Binary number. Truth table is (STEP 1)

minterm	4-bit Gray				4-bit Binary			
	G ₃	G ₂	G ₁	G ₀	B ₃	B ₂	B ₁	B ₀
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	1	0	0	1	0
3	0	0	1	0	0	0	1	1
4	0	1	1	0	0	1	0	0
5	0	1	1	1	0	1	0	1
6	0	1	0	1	0	1	1	0
7	0	1	0	0	0	1	1	1
8	1	1	0	0	1	0	0	0
9	1	1	0	1	1	0	0	1
10	1	1	1	1	1	0	1	0
11	1	1	1	0	1	0	1	1
12	1	0	1	0	1	1	0	0
13	1	0	1	1	1	1	0	1
14	1	0	0	1	1	1	1	0
15	1	0	0	0	1	1	1	1

CODE CONVERSIONS- GRAY TO BINARY

4-bit Gray code to 4-bit Binary code:

(STEP 2)

- ▶ From the truth table, the equation for each output variable in terms of minterms are
 - ▶ $B_3 = \sum m(8, 9, 10, 11, 12, 13, 14, 15)$
 - ▶ $B_2 = \sum m(4, 5, 6, 7, 8, 9, 10, 11)$
 - ▶ $B_1 = \sum m(2, 3, 4, 5, 8, 9, 14, 15)$
 - ▶ $B_0 = \sum m(1, 2, 4, 7, 8, 11, 13, 14)$

- ▶ The reduced SOP expression for B_3 , B_2 , B_1 AND B_0 is calculated using K-map

CODE CONVERSIONS- GRAY TO BINARY

4-bit Gray code to 4-bit Binary code: Using K-maps, the minimal expression for output variables B_3 , B_2 , B_1 , and B_0 , is obtained in terms of input variables, G_3 , G_2 , G_1 , and G_0 . (STEP 3)

		G_1G_0			
		00	01	11	10
G_3G_2	00	0	0	0	0
	01	0	0	0	0
	11	1	1	1	1
	10	1	1	1	1

$$B_3 = G_3$$

		G_1G_0			
		00	01	11	10
G_3G_2	00	0	0	1	1
	01	1	1	0	0
	11	0	0	1	1
	10	1	1	0	0

$$B_1 = (G_3 \text{ XOR } G_2) \text{ XOR } G_1$$

$$= B_2 \text{ XOR } G_1$$

		G_1G_0			
		00	01	11	10
G_3G_2	00	0	0	0	0
	01	1	1	1	1
	11	0	0	0	0
	10	1	1	1	1

$$B_2 = G_3'G_2 + G_3G_2'$$

$$= G_3 \text{ XOR } G_2$$

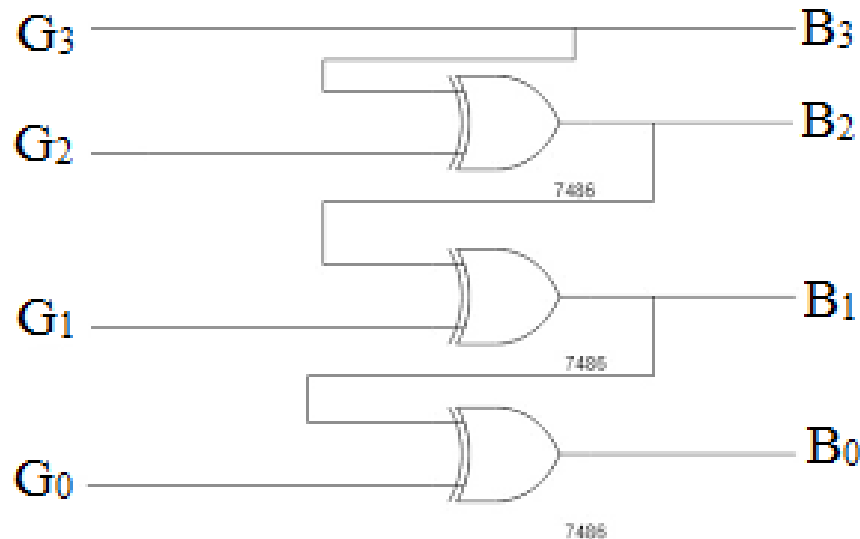
		G_1G_0			
		00	01	11	10
G_3G_2	00	0	1	0	1
	01	1	0	1	0
	11	0	1	0	1
	10	1	0	1	0

$$B_0 = B_1 \text{ XOR } G_0$$

CODE CONVERSIONS- GRAY TO BINARY

- ▶ The logic gate realization of 4-bit Gray to Binary code converter requires 3 XOR gates.
- ▶ The logic diagram for the converter is

(STEP-4)



CODE CONVERSIONS-BCD TO EXCESS-3

4-bit BCD code to 4-bit EXCESS-3 code

- ▶ BCD Code: Four input variables, A, B, C and D
- ▶ EXCESS-3 Code: Four output variables W, X, Y and Z
- ▶ Output in EXCESS-3 code is obtained by adding 3(0011) to the respective input combination
- ▶ The six bit combinations, from decimal 10 to 15 (1010, 1011, 1100, 1101, 1110, 1111) are invalid combinations in BCD code, therefore are considered don't care combinations

CODE CONVERSIONS-BCD TO EXCESS-3

4-bit BCD code to 4-bit EXCESS-3 code:

- The input is 4-bit BCD number and output is 4-bit number in EXCESS-3 code. Truth table is (STEP 1)

minterm	4-bit BCD				4-bit EXCESS-3			
	A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

CODE CONVERSIONS-BCD TO EXCESS-3

4-bit BCD code to 4-bit EXCESS-3 code:

(STEP 2)

- ▶ From the truth table, the equation for each output variable in terms of minterms are
 - ▶ $W(A,B,C,D) = \sum m (5, 6, 7, 8, 9) + d(10, 11, 12, 13, 14, 15)$
 - ▶ $X(A,B,C,D) = \sum m (1, 2, 3, 4, 9) + d(10, 11, 12, 13, 14, 15)$
 - ▶ $Y(A,B,C,D) = \sum m (0, 3, 4, 7, 8) + d(10, 11, 12, 13, 14, 15)$
 - ▶ $Z(A,B,C,D) = \sum m (0, 2, 4, 6, 8) + d(10, 11, 12, 13, 14, 15)$
- ▶ The reduced SOP expression for W, X, Y AND Z is calculated using K-map

CODE CONVERSIONS-BCD TO EXCESS-3

4-bit BCD code to 4-bit EXCESS-3 code: Using K-maps, the minimal expression for output variables W, X, Y, and Z, is obtained in terms of input variables, A, B, C, and D. (STEP 3)

		CD		C		
		00	01	11	10	
A	AB	00				B
	01		1	1	1	
	11	X	X	X	X	
	10	1	1	X	X	
		D				
		W				

		CD		C		
		00	01	11	10	
A	AB	00		1	1	B
	01	1				
	11	X	X	X	X	
	10		1	X	X	
		D				
		X				

		CD		C		
		00	01	11	10	
A	AB	00	1		1	B
	01	1		1		
	11	X	X	X	X	
	10	1		X	X	
		D				
		Y				

		CD		C		
		00	01	11	10	
A	AB	00	1			B
	01	1			1	
	11	X	X	X	X	
	10	1		X	X	
		D				
		Z				

$$W = A + B(C+D)$$

$$X = B'(C+D) + B(C+D)'$$

$$Y = CD + (C+D)'$$

$$Z = D'$$

- The code converter is designed using basic gates

CODE CONVERSIONS-BINARY TO BCD

4-bit Binary to BCD code

- ▶ 4-bit Binary code has input variables B_3 , B_2 , B_1 , and B_0 . Total 16 unique combinations, ranging from decimal equivalent 0 to 15
- ▶ BCD (Binary Coded Decimal) codes comprises of binary equivalent for only decimal digit 0 to 9
- ▶ In this conversion, 1010, 1011, 1100, 1101, 1110, 1111 (10 to 15 decimal equivalent) are represented by two BCD digits i.e. in total 8 bits.
- ▶ Since, first three bits will be 0 for all combinations, two BCD digit is represented by 5-bit output variables A,B,C,D and E.
- ▶ A is the MSB and E is the LSB

Decimal Number	Binary Code	BCD Code
3	0011	0011
9	1001	1001
10	1010	0001 0000
13	1101	0001 0011
15	1111	0001 0101
19	10011	0001 1001

CODE CONVERSIONS-BINARY TO BCD

4-bit Binary to BCD code: The input is a 4-bit Binary number and output is an equivalent BCD number. Truth table is (STEP 1)

minterm	4-bit Binary				BCD				
	B ₃	B ₂	B ₁	B ₀	A	B	C	D	E
0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0	1
2	0	0	1	0	0	0	0	1	0
3	0	0	1	1	0	0	0	1	1
4	0	1	0	0	0	0	1	0	0
5	0	1	0	1	0	0	1	0	1
6	0	1	1	0	0	0	1	1	0
7	0	1	1	1	0	0	1	1	1
8	1	0	0	0	0	1	0	0	0
9	1	0	0	1	0	1	0	0	1
10	1	0	1	0	1	0	0	0	0
11	1	0	1	1	1	0	0	0	1
12	1	1	0	0	1	0	0	1	0
13	1	1	0	1	1	0	0	1	1
14	1	1	1	0	1	0	1	0	0
15	1	1	1	1	1	0	1	0	1

CODE CONVERSIONS-BINARY TO BCD

4-bit Binary to BCD code:

(STEP 2)

- ▶ From the truth table, the equation for each output variable in terms of minterms are
 - ▶ $A = \sum m(10, 11, 12, 13, 14, 15)$
 - ▶ $B = \sum m(8, 9)$
 - ▶ $C = \sum m(4, 5, 6, 7, 14, 15)$
 - ▶ $D = \sum m(2, 3, 6, 7, 12, 13)$
 - ▶ $E = \sum m(1, 3, 5, 7, 9, 11, 13, 15)$

- ▶ The reduced SOP expression for A, B, C, D and E is calculated using K-map

CODE CONVERSIONS-BINARY TO BCD

4-bit Binary to BCD code:

- ▶ Using K-maps, the minimal expression for output variables A, B, C, D, and E, is obtained in terms of input variables, B_3 , B_2 , B_1 , and B_0 (STEP 3)
- ▶ The output equation obtained are
$$A = B_3B_2 + B_3B_1$$
$$B = B_3 B_2'B_1'$$
$$C = B_3'B_2 + B_2B_1$$
$$D = B_3 B_2B_1' + B_3'B_1$$
$$E = B_0$$
- ▶ The code converter is designed using basic gates

ALPHANUMERIC CODES

- ▶ These codes represent numbers, alphabets and other usual symbols
- ▶ This makes the system capable of handling non-numerical information
- ▶ A binary code is assigned to each alphabet. Binary code of n -bit represents 2^n elements
- ▶ Complete and adequate set of necessary characters include:
 - ▶ 26 lower case alphabets
 - ▶ 26 upper case alphabets
 - ▶ 10 numerical digits
 - ▶ 25 special characters, +,*,#,% etc
- ▶ Total 87 symbols are encoded using 7-bit code and an eighth bit is for parity
- ▶ Two types of codes are used
 - ▶ Extended BCD Interchange Code (EBCDIC)
 - ▶ American Standard Code for Information Interchange (ASCII)

ALPHANUMERIC CODES-ASCII

- ▶ American Standard Code for Information Interchange (ASCII)
 - ▶ Standardized code which is widely used
 - ▶ 7-bit unique combination is used to represent numbers, alphabets and other usual symbols.
 - ▶ In total, $2^7 = 128$ unique characters can be encoded
 - ▶ Can be extended to 8-bit representation to encode 128 more characters, i.e. 256 in total

ALPHANUMERIC CODES-ASCII

Ascii	Char	Ascii	Char	Ascii	Char	Ascii	Char
0	Null	32	Space	64	@	96	`
1	Start of heading	33	!	65	A	97	a
2	Start of text	34	"	66	B	98	b
3	End of text	35	#	67	C	99	c
4	End of transmit	36	\$	68	D	100	d
5	Enquiry	37	%	69	E	101	e
6	Acknowledge	38	&	70	F	102	f
7	Audible bell	39	'	71	G	103	g
8	Backspace	40	(	72	H	104	h
9	Horizontal tab	41	)	73	I	105	i
10	Line feed	42	*	74	J	106	j
11	Vertical tab	43	+	75	K	107	k
12	Form feed	44	,	76	L	108	l
13	Carriage return	45	-	77	M	109	m
14	Shift in	46	.	78	N	110	n
15	Shift out	47	/	79	O	111	o
16	Data link escape	48	0	80	P	112	p
17	Device control 1	49	1	81	Q	113	q
18	Device control 2	50	2	82	R	114	r
19	Device control 3	51	3	83	S	115	s
20	Device control 4	52	4	84	T	116	t
21	Neg. acknowledge	53	5	85	U	117	u
22	Synchronous idle	54	6	86	V	118	v
23	End trans. block	55	7	87	W	119	w
24	Cancel	56	8	88	X	120	x
25	End of medium	57	9	89	Y	121	y
26	Substitution	58	:	90	Z	122	z
27	Escape	59	;	91	[	123	{
28	File separator	60	<	92	\	124	
29	Group separator	61	=	93	]	125	}
30	Record separator	62	>	94	^	126	~
31	Unit separator	63	?	95	_	127	Forward del.

ALPHANUMERIC CODES-EBCDIC

- ▶ Extended BCD Interchange Code (EBCDIC)
 - ▶ Widely used in IBM device
 - ▶ Differs from ASCII only in grouping combinations
 - ▶ 8-bit unique combination is used to represent numbers, alphabets and other usual symbols and ninth bit for parity
 - ▶ In total, $2^8 = 256$ in total are encoded

Problems (TBD)

1. Write all 4-bit codes for 2421
2. Write all 4-bit codes for 7421
3. Convert the following into Binary code
 - a) 011
 - b) 010011
 - c) 1110110
 - d) 101010
 - e) 1111111
4. Convert the following in Gray code
 - a) 010
 - b) 1101101
 - c) 111101100